

INSTITUTO FEDERAL

Mato Grosso do Sul

Prog. Disp. Móveis II

Milena Alegre

Configuração inicial:

-  **1. Node.js (LTS):** Garanta que o Node.js está instalado na máquina abrindo o terminal e digitando `node -v`.
-  **2. Visual Studio Code:** Nosso editor principal. Recomendada a extensão *ES7+ React/Redux/React-Native snippets*.
-  **3. Expo Go App:** Baixe o app "Expo Go" na Play Store (Android) ou App Store (iOS) do seu celular.
-  **4. Conexão de Rede:** Computador e celular devem estar conectados na mesma rede Wi-Fi (ou via cabo USB com adb).
-  **5. Copilot:** Construção mais rápida de aplicações baseadas no modelo de negócio



Kit essencial

Por que React Native e Expo?

-  **Desenvolvimento Agilizado:** O mesmo código funciona perfeitamente em iOS, Android e Web.
-  **Leveza de Máquina:** Evitamos emuladores pesados usando o Expo Go diretamente no celular físico.
-  **Fast Refresh:** Alterações no código são refletidas instantaneamente sem recompilar.
-  **Mercado Aquecido:** Alta demanda por desenvolvedores JavaScript/TypeScript Fullstack.



Mudanças na Instalação de APKs

Instalar APKs diretamente por fora das lojas está se tornando cada vez mais restrito no ecossistema Android por motivos de segurança.

Na disciplina, entenderemos como o **EAS Build (Expo Application Services)** compila nosso projeto na nuvem para gerar os pacotes oficiais aceitos pela Google Play Store e Apple App Store.

- **Exigência de Cadastro:** A partir de **30 de setembro de 2026**, o Android passará a aplicar novas regras de segurança no Brasil e em mais alguns países, exigindo que aplicativos externos venham de um desenvolvedor com identidade verificada.
- **Bloqueio de Fontes Anônimas:** Instaladores de fontes totalmente desconhecidas ou sem registro válido enfrentarão fortes barreiras ou bloqueios automáticos pelo sistema para evitar vírus e malwares.
- **Avisos e Etapas Extras:** O sistema exibirá alertas de risco mais claros e fluxos avançados para quem tentar instalar arquivos de origem não confirmada.

Comandos no Terminal

1. Criar novo projeto:

```
npx create-expo-app meu-app
```

2. Entrar na pasta do projeto:

```
cd meu-app
```

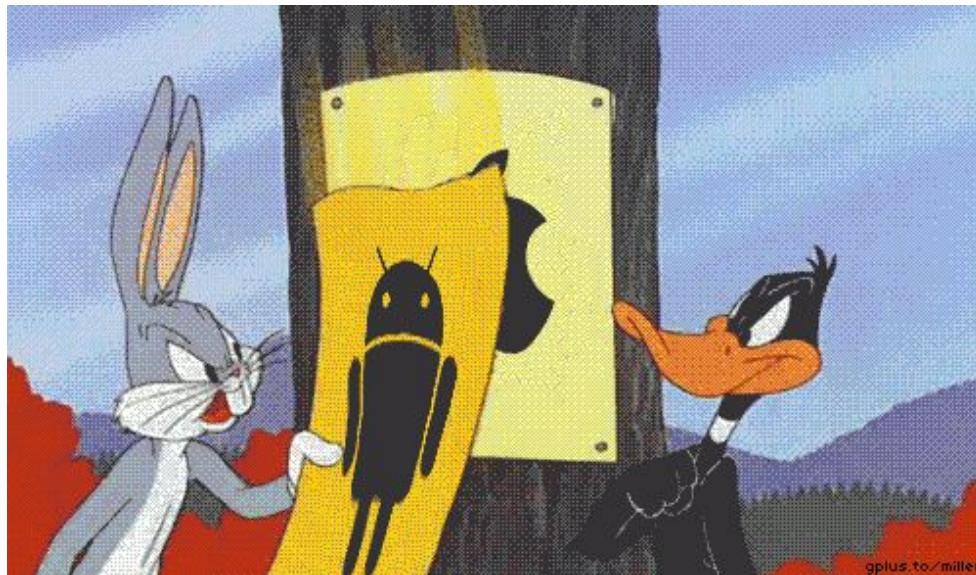
Conectando ao Aparelho

- Um **QR Code** será exibido no terminal.
- Abra o aplicativo **Expo Go** no celular.
- Selecione *"Scan QR Code"* e aponte para a tela.
- Seu app será carregado e atualizado instantaneamente a cada salvamento no VS Code!

Relembrando

React	React Native
Cria sites	Cria aplicativos Android/iOS
Usa HTML (<code>div</code> , <code>button</code> , <code>input</code>)	Usa componentes (<code>View</code> , <code>Text</code> , <code>TextInput</code> , <code>Button</code>)
Roda no navegador	Roda no celular
CSS	StyleSheet (parecido com CSS)

Relembrando



Aquecendo

Projeto de conversor (Dólar para Real)

Usamos o hook `useState` para capturar o texto digitado e armazenar o resultado convertido.

Passo 1: Criar o Projeto Expo

Passo 2: Programar o Contador Interativo

Criar os Estados (`useState`)

Implementar a Lógica e a Interface no `App.js`

Variáveis: `valorEntrada`, `cotacao`, `resultado`



Desafio Extra:

- Adicionar alguma validação adicional, exemplo: alerta para valor vazio, botão de trocar de real -> dolar/ dolar->real

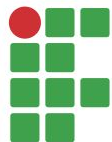
Aquecendo

keyboardType="numeric": No mobile, se você não colocar essa propriedade no `<TextInput>`, o celular abre o teclado alfa-numérico normal (o que é ruim para UX quando se espera apenas números).

onChangeText={setValorReal}: Na Web (React JS), costuma-se usar `e.target.value`. No React Native, o `onChangeText` já devolve direto a string digitada, sem precisar acessar o evento `e`.

parseFloat() e **toFixed(2)**: Como tudo que vem de um `TextInput` é **string**, precisamos converter para número com `parseFloat()` antes de fazer operações matemáticas, e usar `toFixed(2)` para formatar como valor de moeda.

milena.alegre@ifms.edu.br



INSTITUTO FEDERAL
Mato Grosso do Sul